Steering through a scanned living room with a dish of neurons

A closed-loop navigation experiment for the Cortical Labs CL1

Alara Dirik

Outlier Labs Biocomputing Hack, November 2026 (working notes)

Abstract

DishBrain showed that cortical cultures on a multi-electrode array improve at Pong when the ball's position is delivered as electrical stimulation and the outcome of each rally is fed back as either predictable or unpredictable stimulation [11]. This document describes a closed-loop experiment that takes the same idea from a one-dimensional paddle to navigation in a real 3D scene. A disc-shaped agent drives at constant speed through the ICL-NUIM living room, a synthetic indoor scene distributed as a point cloud [4, 9]. The point cloud is converted into an obstacle map, from which eight range rays are cast around the agent. Each ray's distance sets the pulse rate (4 Hz to 40 Hz) on one of eight sensory electrodes of the CL1's 8×8 array. Spiking in two motor regions at the edges of the array turns the agent left or right. Progress towards the goal and reaching it earn predictable stimulation; collisions earn four seconds of unpredictable stimulation. The experiment includes the silent-feedback and rest controls from DishBrain, a mirrored-layout control, and a toy culture for the cl-sdk simulator that tests the whole encode–decode chain before any cells are involved. On the simulator, a toy culture wired as a Braitenberg avoider steers away from walls (steering coupling $r \approx -0.5$) and gets about six times further along the route than a toy culture whose responses never reach the motor regions. Neither shows a trend across episodes, which is correct because the toy culture cannot learn. The checks also caught a decoder design that produced spurious "learning" on this non-plastic culture. No results on living cells are reported yet.

1 Introduction

Biological neural networks learn from very few samples and use little energy, which has renewed interest in using living neurons as computing substrates [17]. The Cortical Labs CL1 makes this experiment programmable. Neurons grow on a 64-electrode array, and a Python API (`cl`) reads detected spikes and schedules current-controlled stimulation in a loop that runs at up to 25 kHz [5, 6]. In principle, anyone who can write a game loop can embody a culture.

The best-known embodied culture is DishBrain [11]. The ball's position was place- and rate-coded onto eight sensory electrodes, spikes in predefined motor regions moved the paddle, and each hit was followed by a predictable stimulus while each miss was followed by an unpredictable one. Cultures given this feedback improved within a session; cultures given no feedback or no sensory input did not improve in the same way. The design is motivated by the free-energy principle [8]: a network that can act on its inputs should come to avoid the actions that lead to unpredictable stimulation.

Pong is a one-dimensional control problem with a dense reward. This project asks the next question on a hackathon timescale: *does the same closed loop work when the world is a real 3D scene and the task is navigation?* Navigation is harder in three ways. The input describes a 2D layout rather than one coordinate. A good action depends on obstacles all around the agent. And the goal is out of sight, so reward comes only through feedback about progress.

The deliverable is a complete, runnable pipeline rather than a result:

- a scene converter that turns the ICL-NUIM point cloud into an obstacle grid, a drivable grid and a geodesic distance field (`prepare_scene.py`);

- a closed loop for the CL1 with a DishBrain-style encoder, a baseline-normalised decoder and a feedback state machine with three conditions (`run.py`, `world.py`);
- a toy culture for the cl-sdk simulator that responds to stimulation, giving a positive and a negative control for the pipeline (`culture.py`);
- analysis of outcomes, within-session learning, condition differences, sensorimotor coupling and stimulation artifacts (`analyze.py`), and a live view (`visualiser/`).

2 Related work

Embodied cultures. Closing the loop between a cultured network and a body is older than the CL1. In the neurally controlled animat, activity in a rat cortical culture moved a simulated animal, and the animal's movement was fed back as stimulation [7]. Novellino et al. [14] connected cultures to a wheeled robot: left and right infrared sensors were rate-coded onto stimulation, and firing in two groups of electrodes set the two wheel speeds, producing obstacle avoidance. This is the closest precedent for the sensor layout used here. Chao et al. [3] trained an embodied culture towards goal-directed movement with patterned training stimulation chosen by a feedback controller, and Warwick et al. [18] drove a mobile robot from a culture. These systems mostly relied on a hand-designed sensorimotor mapping or explicit training stimulation, not on predictable versus unpredictable feedback.

Learning from structured feedback. Shahaf and Marom [16] showed that cultures can learn a stimulus–response mapping when stimulation stops as soon as the desired response appears, and Isomura et al. [10] showed that cultured networks separate mixed sources in a way consistent with free-energy minimisation. DishBrain combined these ideas in a game: predictable stimulation after a hit, unpredictable stimulation after a miss [11]. A follow-up compared the cultures with deep reinforcement learning agents given the same number of episodes and found the cultures more sample efficient [12]. Most recently, Robbins et al. [15] embodied cortical organoids in the cart-pole task. They delivered a training signal only when performance stopped improving, chose which electrodes received it with a reinforcement learning algorithm, and found that this adaptive training clearly outperformed random training. The improvement was short-lived: it had faded after 45 minutes of rest.

Cultures as reservoirs. A different line of work treats the culture as a fixed nonlinear system and trains only a readout: Brainoware used an organoid to recognise speakers from audio [2]. On the CL1, Loeffler et al. [13] classified spatio-temporal MNIST digits with human iPSC-derived cultures. They found that modular cortical networks did best, and that without strict artifact control and trial-based cross-validation, stimulation artifacts and temporal leakage can produce false positives. This experiment adopts two lessons from that work: motor electrodes are physically separated from stimulated ones, and the analysis checks directly whether stimulation reaches the motor electrodes (Section 3.8).

The scene. ICL-NUIM is a synthetic living room and office built for benchmarking RGB-D odometry and reconstruction [9]; the augmented version distributed with Open3D [4, 19] includes a dense point cloud of the room, which is used here.

3 Method

Figure 1 summarises the loop. All constants live in `config.py`; the values below are the defaults.

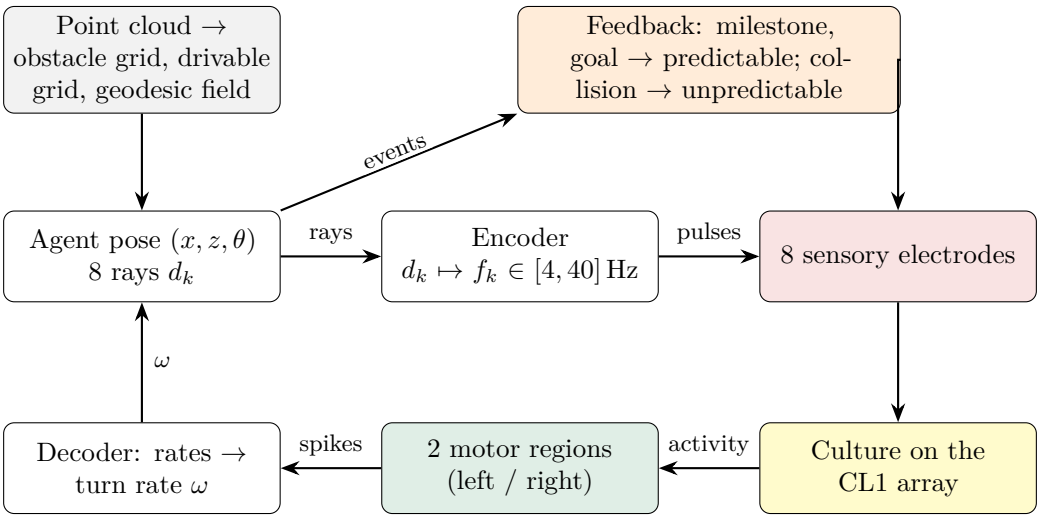


Figure 1: The closed loop, run at 100 Hz. Everything outside the yellow box is Python on the CL1; the culture only ever sees pulses on eight electrodes.

3.1 From point cloud to navigation map

The augmented ICL-NUIM living room point cloud (2.66 M points, y up) is downloaded from the Open3D release mirror and checked against its MD5 checksum. Points are cropped to the room interior ($x \in [-2.70, 2.75]$, $z \in [-2.05, 3.20]$ m), and heights are measured from the 1st percentile of y (the floor). Three grids with 5 cm cells are built (Figure 2):

- **obstacles**: cells containing any point 0.30 m to 1.20 m above the floor (furniture and walls), followed by a morphological closing. The cloud contains two full floor-level planes about 0.22 m apart (probably the floor surface and the slab under it), and the lower bound keeps both out of the grid;
- **drivable**: cells where the floor was observed and whose distance to the nearest obstacle exceeds the agent radius of 0.20 m, keeping the largest connected region (7.1 m^2);
- **geodesic distance** to the goal, computed with Dijkstra's algorithm on the 8-connected drivable grid.

The agent starts in the alcove in the bottom-right corner of the map at $(1.85, -1.30)$, facing $+z$, with a random heading offset of up to $\pm 22.5°$ each episode. The goal is the far corner at $(-0.75, 2.65)$, which is 5.14 m away along the shortest path and reachable either way round the coffee table. This 2.5D projection discards height above the agent. Taking the full 3D structure into account is on the to-do list. The map is drawn with z pointing up, which mirrors it relative to a true top-down view. This does not matter, because the agent lives entirely in map coordinates.

3.2 Agent and sensing

The agent moves forward at a constant $v = 0.2\,\mathrm{m\,s^{-1}}$; the culture chooses only its turn rate ω. At each 10 ms tick, $\theta \leftarrow \theta + \omega\,\Delta t$ and the position advances by $v\,\Delta t$ along θ. A step that would put the centre outside the drivable grid is a collision. Eight rays are cast at $\theta + 22.5° + 45°k$, $k = 0, \ldots, 7$, so that rays 0–3 sweep the left side from front to back and rays 4–7 sweep the right side from back to front. Each ray is marched in 2.5 cm steps until it reaches an obstacle cell or the maximum range $d_{\max} = 2$ m. There is deliberately no ray straight ahead, so every ray belongs unambiguously to the left or the right half of the field of view.

3.3 Electrode layout

Figure 3 shows the layout. Like DishBrain, it uses eight sensory electrodes and two counterbalanced motor regions. The motor regions sit at opposite edges, as far as possible from the stimulated

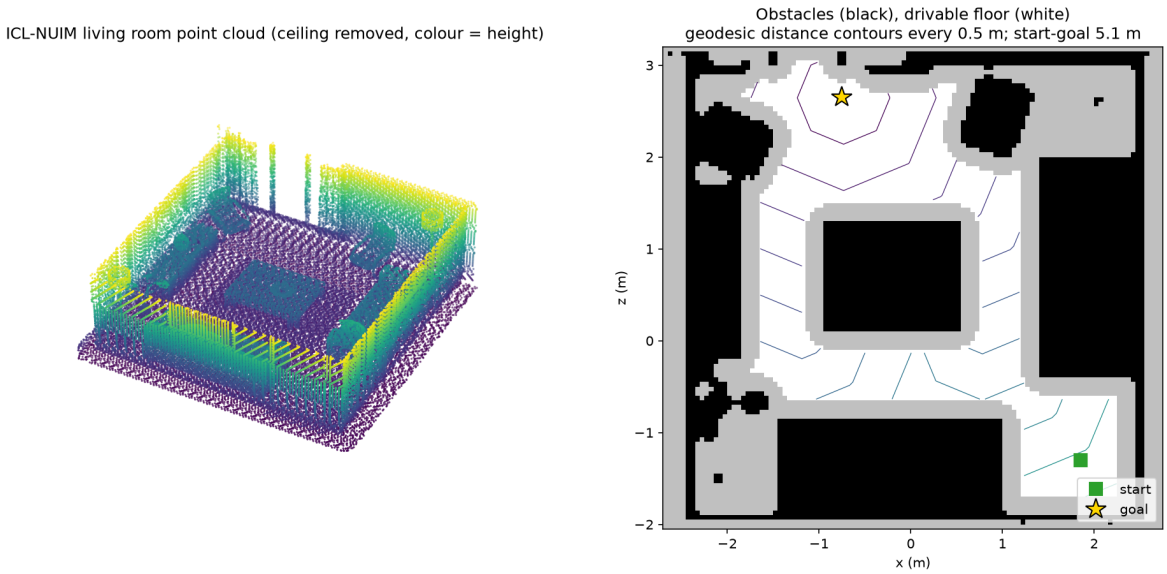


Figure 2: Left: the room's point cloud. Right: the navigation map derived from it. Grey cells are floor that the agent's centre cannot reach without touching an obstacle. Contours show the geodesic distance to the goal (star), which defines progress and milestones.

electrodes, so that most of what they record should come from activity travelling through the culture. The placement implies a topographic prior: obstacles on the left stimulate the left half of the sensory strip, which is closer to the left motor region. If activity spreads locally, this alone could push the agent *towards* obstacles. The mirrored condition (Section 3.7) controls for this kind of wiring effect.

3.4 Sensory encoding

Each ray's distance is converted into a pulse rate on its own electrode, as in the rate code DishBrain used for ball distance:

$$f_k = f_{\min} + (f_{\max} - f_{\min})\,(1 - \min(d_k/d_{\max},\,1))\,, \qquad f_{\min} = 4\,\text{Hz},\;\; f_{\max} = 40\,\text{Hz}. \tag{1}$$

Pulses are timed by one phase accumulator per electrode: $\phi_k \mathrel{+}= f_k \Delta t$, and a pulse is due when $\phi_k \geq 1$. The accumulators start at random phases so that the electrodes do not fire in lockstep. All pulses due in a tick go out in one `neurons.stim(ChannelSet(...), StimDesign(160,-1.0,160,1.0))` call: a charge-balanced biphasic pulse, 160 µs per phase at 1 µA, negative phase first. DishBrain specified its stimulation in millivolts on a different array, so these currents are a starting point to confirm with the CL1 team, not a translation.

3.5 Motor decoding

At each tick, the spikes detected on the two motor regions are counted and smoothed into rates r_L, r_R with a 100 ms exponential moving average. Each rate is divided by its own slow baseline b_s, a 30 s moving average of the rate. The turn rate is then set by the lateralisation index of the normalised rates:

$$\tilde{r}_s = \frac{r_s}{b_s}, \qquad \omega = \omega_{\max} \tanh\!\left(g\, \frac{\tilde{r}_L - \tilde{r}_R}{\tilde{r}_L + \tilde{r}_R} \right), \qquad \omega_{\max} = 1.5\,\text{rad}\,\text{s}^{-1},\;\; g = 6. \tag{2}$$

The baseline removes each region's own bias, so a region that is simply more active does not steer the agent. The index makes the turn rate depend only on how activity is split between the regions, not on its overall level.

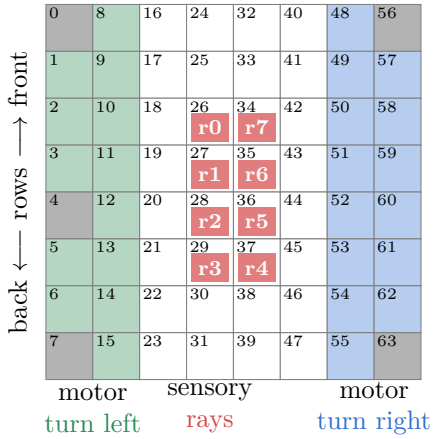


Figure 3: Channel layout on the CL1's 8×8 array (channel $= 8 \cdot \text{column} + \text{row}$). Grey channels are unusable: the four grounded corners and the reference channel 4. The sensory strip is a coarse top-down map of the agent's view, with left rays in column 3 and right rays in column 4. Columns 2 and 5 are a buffer that keeps stimulation artifacts and direct evoked responses away from the 13 left and 14 right motor electrodes.

event	`feedback` condition	`silent` / `rest`	then
milestone (0.5 m of new geodesic progress)	predictable: all 8 sensory, 10 pulses @ 100 Hz	100 ms pause	continue
goal (within 0.3 m)	predictable (as above)	100 ms pause	new episode
collision	unpredictable: random sensory electrode, 2 µA, 5 Hz for 4 s	4 s pause	new episode
timeout (60 s)	none	none	new episode

Table 1: Events and their consequences. During feedback and pauses the agent is frozen and no sensory pulses are sent, so feedback replaces sensory input as in DishBrain.

This second step matters. An earlier version used the plain difference $\tilde{r}_L - \tilde{r}_R$. When sensory stimulation started after calibration, both regions sped up several-fold, the difference saturated, and the agent turned at full rate for the first minute or so, until the baselines caught up. The resulting improvement over episodes was statistically significant ($\rho = 0.54$, $p = 0.002$) on a toy culture with no plasticity at all. Analysed naively, it would have looked like learning. Before the first episode there is a 10 s calibration period without stimulation, which fills the baselines. The session stops with an error if either motor region is silent during calibration, because the electrodes would then need to be moved to active areas of the culture.

3.6 Feedback and episodes

Table 1 lists the events. The feedback parameters follow DishBrain: a 100 ms, 100 Hz burst on all sensory electrodes after a hit, and 4 s of 5 Hz pulses at random sensory sites, at twice the hit amplitude, after a miss [11, 12]. Pong produces a hit every few seconds, but reaching the goal is rare early on, so progress milestones provide denser predictable feedback. A milestone fires each time the geodesic distance to the goal drops 0.5 m below the best value reached so far in the episode. This rewards new progress only, not driving back and forth.

The culture is never told where the goal is. The rays carry only obstacle information, so a culture can learn to avoid collisions from the rays alone. Heading for the goal requires learning a route from feedback alone, which is much harder. Adding an explicit goal-bearing input is the first extension on the to-do list.

3.7 Experimental design

Three conditions follow the DishBrain controls:

`feedback` sensory stimulation plus predictable and unpredictable feedback;
`silent` sensory stimulation, with each feedback period replaced by a silent pause of the same length. Any improvement here comes from the sensory input alone;
`rest` no stimulation at all; spontaneous activity steers the agent. This measures what an unstimulated culture does in this room.

The `-mirror` flag swaps the motor regions. If behaviour comes from the layout's topographic prior, it flips with the mirror. If it is learned from feedback, it should appear in both orientations. The recommended unit is a 20-minute session, as in DishBrain. Conditions should be counterbalanced in order across sessions and cultures, with at least three sessions per condition before comparing conditions.

3.8 Measurements and analysis

Every tick is logged to a `nav` data stream: pose, rays, turn rate, motor rates and delivered pulses. Every episode start and end is logged to an `events` data stream. Both are stored in the CL HDF5 recording next to the spikes and stimulation events. `analyze.py` reports:

- **progress** per episode, $1 - D_{\text{end}}/D_{\text{start}}$ with D the geodesic distance (1 at the goal), and the fraction of episodes ending at the goal, in a collision or in a timeout;
- **within-session learning**: the Spearman correlation between episode index and progress, and a one-sided Mann–Whitney test of the last third of episodes against the first third. DishBrain compared the first 5 minutes with the last 15;
- **between conditions**: two-sided Mann–Whitney tests on per-episode progress, pooled by condition. Episodes within a session are not independent, so these p-values are optimistic; a session-level or mixed-effects analysis is on the to-do list;
- **steering coupling**: the correlation, over all ticks of play, between how much closer the obstacles are on the left than on the right ($\sum_{k<4} c_k - \sum_{k\geq 4} c_k$ with closeness $c_k = 1 - \min(d_k/d_{\max}, 1)$) and the decoded turn rate. Negative means the agent turns away from obstacles, positive means it turns towards them, and near zero means motor output ignores the sensory input;
- **motor PSTH**: the firing rate of the motor electrodes from −20 ms to 30 ms around each sensory pulse, in 1 ms bins. A sharp peak in the first 2 ms indicates a stimulation artifact rather than a neural response. This should be checked before interpreting any steering. Because pulses are issued on the 10 ms tick grid, dense stimulation produces a periodic PSTH; a short session with sparse pulses gives a cleaner picture.

3.9 Implementation

The loop runs at 100 Hz with `neurons.loop(100)`. Each tick casts the rays (vectorised NumPy, well under a millisecond), updates the decoder and state machine, issues at most one `stim` call, and appends one data-stream entry. On the CL1, the loop raises an error if a tick overruns, and the code relies on that rather than recovering silently. The simulator does not enforce this check (cl-sdk 1.0.0 prints a warning that jitter detection is unsupported), so timing problems will only show up on the device. `prepare_scene.py` runs once on a laptop. Only `scene/livingroom.npz` (13 kB) needs copying to the CL1, which needs just `numpy` and `cl` to run `run.py`. Passing `live_view=True` to `run_session` shows a top-down view of the room next to the electrode array in Jupyter (Figure 4).

3.10 Pipeline checks on the simulator

The SDK's default random source fires spontaneously but ignores stimulation. On that source, a correct pipeline and a broken one look the same. `culture.py` replaces it with a toy culture that adds evoked spikes on top of the random background:

- **local**: each pulse makes electrodes within 1.5 pitches fire with probability 0.4 (scaled by current), 2–10 ms later. These responses never reach the motor regions, so this is the negative

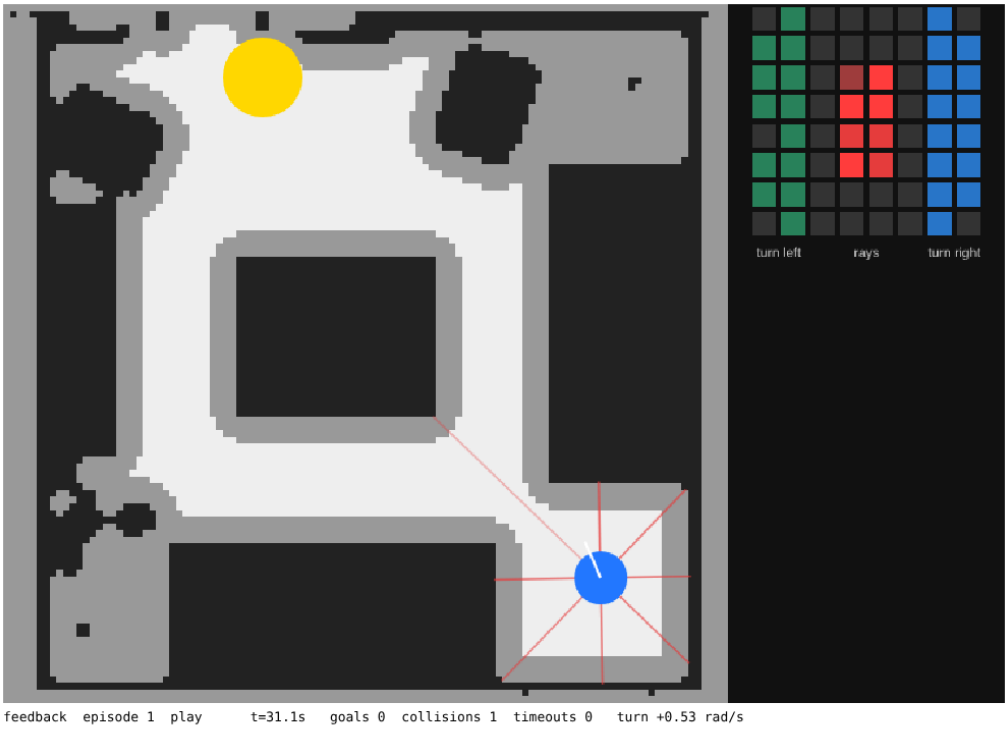


Figure 4: The live view during a simulator session. The agent (blue) faces the white tick; red rays are brighter when an obstacle is closer, the green line is the current episode's path, and the yellow disc is the goal. On the array panel, sensory electrodes flash when pulsed and motor regions glow with their current rate.

control;

- **avoider**: in addition, each left-ray electrode drives every right-motor electrode (and vice versa) with probability 0.5, 5–20 ms later. This is a Braitenberg vehicle [1] that turns away from close obstacles. It is a positive control: if the encoder, layout, decoder and sign conventions are right, it must avoid walls.

The toy culture has no plasticity, so no learning is expected, and none should be detected.

Table 2 and Figure 5 show the expected pattern. The avoider turns away from obstacles (coupling −0.50). It leaves the alcove, follows the right-hand corridor north and usually collides where the corridor opens out at the top of the map, about halfway along the route. That is about six times as far as the local toy culture (0.48 vs 0.08 progress, $p < 10^{-13}$). The local toy culture and the rest condition do not differ ($p = 0.61$), and neither shows any coupling. This shows that sensory pulses that never reach the motor regions are equivalent to no input at all. The mirrored avoider still avoids, because its wiring was built against the decoded layout. Most importantly, no session shows a significant trend over episodes, which is correct for a culture with no plasticity. On real cells, the avoider's numbers are a rough upper reference for what pure reflexive avoidance can achieve in this room, and the local and rest sessions show what chance looks like. None of these simulator sessions reached the goal: reflexes alone do not find the far corner.

4 To-do list

Before the event

☒ Scene converter, closed loop, toy culture, analysis and live view; pipeline checks on the simulator (Table 2).

session	episodes	goal	collision	progress	first	last	ρ (p)	coupling
`feedback`, avoider	31	0.00	1.00	0.48	0.53	0.48	−0.20 (0.28)	−0.50
`feedback -mirror`, avoider	31	0.00	1.00	0.42	0.50	0.41	−0.07 (0.72)	−0.51
`feedback`, local	69	0.00	1.00	0.08	0.09	0.08	0.01 (0.95)	−0.04
`rest`, local	71	0.00	1.00	0.07	0.08	0.08	0.04 (0.74)	−0.03

Table 2: Four 10-minute sessions on the simulator (accelerated time, seed 7). "first" and "last" are the mean progress of the first and last third of episodes; ρ is the Spearman correlation of progress with episode index; "coupling" is the steering coupling defined in Section 3.8.

- ☐ Set up a Python 3.12+ environment on the laptop, `pip install -r requirements.txt`, run `prepare_scene.py`, and repeat the `-toy avoider` and `-toy local` runs to confirm everything works on the laptop.
- ☐ Questions for the Callosum / Frontier Computing team: how code reaches the CL1 (Jupyter on the device?), which cell type is plated and how old the culture is, sensible stimulation currents (the 1 µA and 2 µA defaults are guesses), and how many minutes of device time each team gets.
- ☐ Re-read the DishBrain methods [11], especially the sensory and motor region placement and the rest periods between rallies.
- ☐ Prepare a one-slide pitch: the loop diagram (Figure 1), the room (Figure 2) and a recording of the live view.

At the event (limited CL1 time)

- ☐ Copy the repository and `scene/livingroom.npz` to the device; no download is needed there.
- ☐ Sanity check as in `CL-00 Hello, Neurons`: pulse one sensory electrode and confirm a response in the visualiser.
- ☐ Look at spontaneous activity across the array. If a motor region is quiet, move `MOTOR_LEFT` and `MOTOR_RIGHT` (and if needed `SENSORY`) in `config.py` to active areas, keeping the buffer columns between them.
- ☐ Run a 2-minute `rest` session and a 2-minute `feedback` session and check the motor PSTH. An early peak means an artifact: lower the current, widen the buffer, or ignore motor spikes within 2 ms of a pulse.
- ☐ Run one `feedback` session (10–20 min) with the live view, then one `rest` session as a baseline, and run `analyze.py` on both.
- ☐ Record the screen of the live view for the pitch.

If there is a day on the CL1

- ☐ Full design: `feedback`, `silent`, `rest`, each at least three 20-minute sessions, counterbalanced, plus `feedback -mirror`; more than one culture if possible.
- ☐ Any change to the decoder or the feedback should first be run on `-toy avoider` and `-toy local` for the session length used on the cells. Any trend across episodes on the toy culture is an artifact of the pipeline, not learning (see the decoding section).
- ☐ Replace pooled episode tests with session-level statistics (one value per session) or a mixed-effects model with episodes nested in sessions nested in cultures.

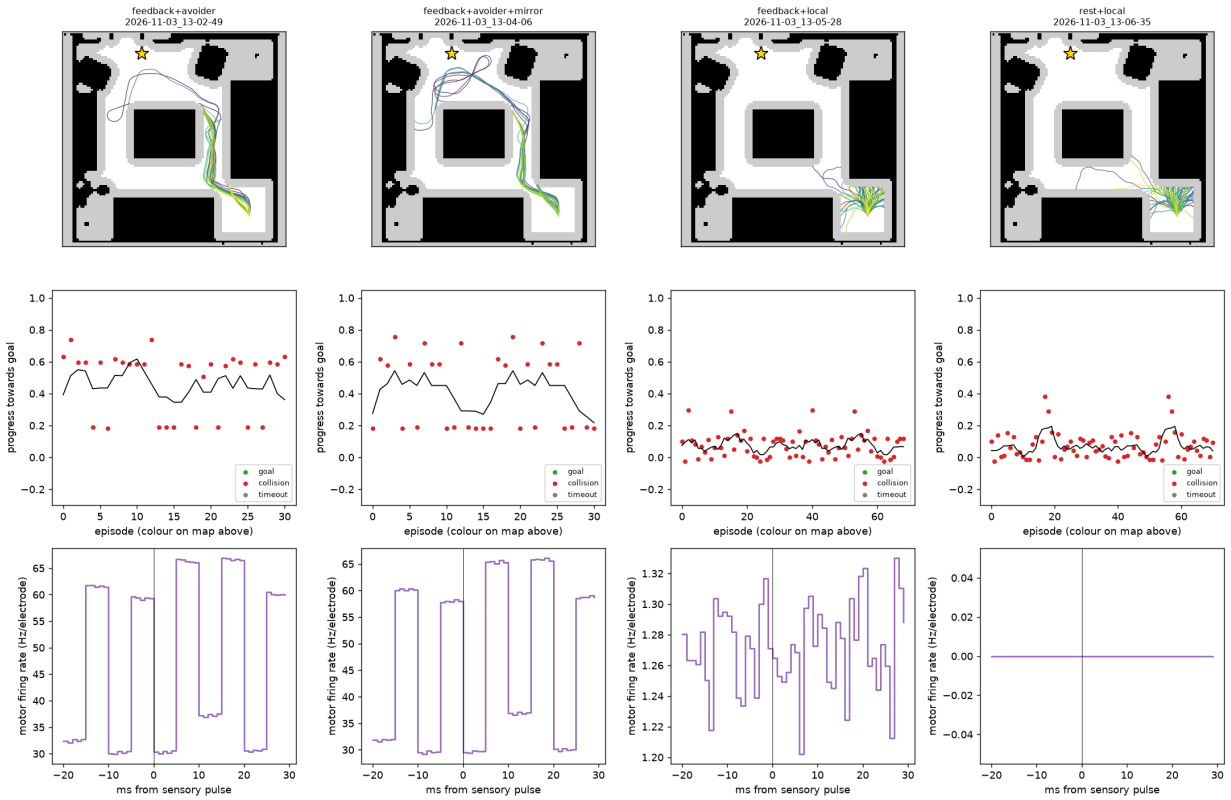


Figure 5: Output of `analyze.py` for the simulator sessions in Table 2. Top: trajectories coloured by episode. Middle: progress per episode, coloured by outcome, with a 5-episode running mean. Bottom: motor PSTH around sensory pulses. For the avoider, the alternating bands come from dense pulses on the 10 ms tick grid combined with the toy's 5–20 ms response latency. For the local toy culture, motor rates stay at the spontaneous level of about 1.3 Hz. The rest condition sends no pulses, so its PSTH is empty.

- ☐ Retention: repeat a `feedback` session after 45–60 minutes of rest, since Robbins et al. [15] found the improvement faded.
- ☐ Software baselines with the same number of episodes: a random-turn agent, a hand-wired Braitenberg avoider, and a small RL agent [12].

Extensions

- ☐ **Goal bearing.** Add one or two electrodes whose rate encodes the direction to the goal (or to the next waypoint on the geodesic path). Without this input the culture can only learn to avoid obstacles.
- ☐ **Curriculum.** Start episodes closer to the goal and move the start back as progress improves; shorter milestones early on.
- ☐ **Adaptive training signal.** Choose which electrodes receive feedback with a bandit or RL algorithm, as in Robbins et al. [15].
- ☐ **Full 3D sensing.** Cast rays at several elevations directly against the point cloud (or a voxel grid) so that table tops and overhangs matter; try the Redwood office scenes or a maze.
- ☐ **Richer decoding.** Use frequency content of the raw signal rather than spike counts, which worked better for digit classification [13]; or choose motor regions per culture, as DishBrain did in pilot experiments.

☐ **A plastic toy culture.** Add spike-timing-dependent plasticity to `culture.py` to test, on the simulator, whether this feedback scheme can shape behaviour at all.

Known limitations

- The toy culture has no plasticity; simulator checks validate the plumbing, not learning.
- Stimulation currents are not calibrated, and DishBrain's voltages do not translate directly to the CL1's current-controlled stimulation.
- In accelerated mode the simulator gives its data producer 100 ms per tick. On a busy two-core machine this occasionally raised `TimeoutError: Timeout waiting for timestamp`; rerunning with nothing else running fixed it.

References

[1] Valentino Braitenberg. *Vehicles: Experiments in Synthetic Psychology*. MIT Press, 1984.

[2] Hongwei Cai, Zheng Ao, Chunhui Tian, Zhuhao Wu, Hongcheng Liu, Jason Tchieu, Mingxia Gu, Ken Mackie, and Feng Guo. Brain organoid reservoir computing for artificial intelligence. *Nature Electronics*, 6:1032–1039, 2023.

[3] Zenas C. Chao, Douglas J. Bakkum, and Steve M. Potter. Shaping embodied neural networks for adaptive goal-directed behavior. *PLoS Computational Biology*, 4(3):e1000042, 2008. doi: 10.1371/journal.pcbi.1000042.

[4] Sungjoon Choi, Qian-Yi Zhou, and Vladlen Koltun. Robust reconstruction of indoor scenes. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5556–5565, 2015.

[5] Cortical Labs. CL API documentation (pre-release notebooks). https://github.com/Cortical-Labs/cl-api-doc, 2025.

[6] Cortical Labs. cl-sdk 1.0.0: Simulator for the Cortical Labs CL API. https://pypi.org/project/cl-sdk/, 2026.

[7] Thomas B. DeMarse, Daniel A. Wagenaar, Axel W. Blau, and Steve M. Potter. The neurally controlled animat: Biological brains acting with simulated bodies. *Autonomous Robots*, 11(3): 305–310, 2001.

[8] Karl Friston. The free-energy principle: A unified brain theory? *Nature Reviews Neuroscience*, 11(2):127–138, 2010.

[9] Ankur Handa, Thomas Whelan, John McDonald, and Andrew J. Davison. A benchmark for RGB-D visual odometry, 3D reconstruction and SLAM. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 1524–1531, 2014.

[10] Takuya Isomura, Kiyoshi Kotani, and Yasuhiko Jimbo. Cultured cortical neurons can perform blind source separation according to the free-energy principle. *PLoS Computational Biology*, 11(12):e1004643, 2015.

[11] Brett J. Kagan, Andy C. Kitchen, Nhi T. Tran, Forough Habibollahi, Moein Khajehnejad, Bradyn J. Parker, Anjali Bhat, Ben Rollo, Adeel Razi, and Karl J. Friston. In vitro neurons learn and exhibit sentience when embodied in a simulated game-world. *Neuron*, 110(23): 3952–3969, 2022. doi: 10.1016/j.neuron.2022.09.001.

[12] Moein Khajehnejad, Forough Habibollahi, Aswin Paul, Adeel Razi, and Brett J. Kagan. Biological neurons compete with deep reinforcement learning in sample efficiency in a simulated gameworld. arXiv:2405.16946, 2024.

[13] Alon Loeffler, Forough Habibollahi, Kwaku Dad Abu-Bonsrah, Azin Azadi, Candice Desouza, Hui Wen Chan, Yusei Nishi, Johnson Zhou, Finn Doensen, Hideaki Yamamoto, Brad Watmuff, and Brett J. Kagan. Handwritten digit classification with neural cultures is influenced by neural architecture, network dynamics, and decoding methods. bioRxiv preprint, 2026.

[14] Antonio Novellino, Paolo D'Angelo, Laura Cozzi, Michela Chiappalone, Vittorio Sanguineti, and Sergio Martinoia. Connecting neurons to a mobile robot: An in vitro bidirectional neural interface. *Computational Intelligence and Neuroscience*, 2007:12725, 2007. doi: 10.1155/2007/12725.

[15] Ash Robbins et al. Goal-directed learning in cortical organoids. *Cell Reports*, 2026. doi: 10.1016/j.celrep.2026.116984.

[16] Goded Shahaf and Shimon Marom. Learning in networks of cortical neurons. *Journal of Neuroscience*, 21(22):8782–8788, 2001.

[17] Lena Smirnova and Thomas Hartung. Neuronal cultures playing Pong: First steps toward advanced screening and biological computing. *Neuron*, 110(23):3855–3856, 2022. doi: 10.1016/j.neuron.2022.11.010.

[18] Kevin Warwick, Dimitris Xydas, Slawomir J. Nasuto, Victor M. Becerra, Mark W. Hammond, Julia H. Downes, Simon Marshall, and Benjamin J. Whalley. Controlling a mobile robot with a biological brain. *Defence Science Journal*, 60(1):5–14, 2010. doi: 10.14429/dsj.60.11.

[19] Qian-Yi Zhou, Jaesik Park, and Vladlen Koltun. Open3D: A modern library for 3D data processing. arXiv:1801.09847, 2018.